B1 Project setup

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B1 Project setup 3
B1-L1 One pyproject.toml and a src layout 3
B1-L2 One virtual environment per project, and python -m pip 8
B1-L3 ruff and mypy in daily work 12
Track B best-practice checklist 17
Appendix: solutions and answer keys 18

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B1 Project setup

Roadmap: Prerequisite for all months. pyproject.toml, virtual environments, ruff, mypy and folder layout.

B1-L1 One pyproject.toml and a src layout

Concept

A data science project starts as a notebook and a CSV. By month three it has helper modules, tests, a model, and three people asking how to run it. The fix is structure from day one.

`pyproject.toml` is the one configuration file for a modern Python project. It holds:

- `[project]`: name, version, `requires-python`, and runtime `dependencies`.
- `[dependency-groups]` (PEP 735): named groups for development only, such as `test` or `lint`. They never ship with the package. Install one with `pip install --group test` (pip 25.1 and later).
- `[tool.*]` tables for tools: `[tool.ruff]`, `[tool.mypy]`, `[tool.pytest.ini_options]`.

One file replaces `setup.py`, `requirements.txt`, `setup.cfg`, `.flake8` and `pytest.ini`.

The src layout puts importable code under `src/`:

```
reach_model/
├── pyproject.toml
├── src/reach_model/        # importable package
│   ├── __init__.py
│   ├── features.py
│   └── train.py
├── tests/                  # pytest, imports reach_model
├── notebooks/              # exploration; imports reach_model, holds no
  logic
└── data/                   # small samples only; real data stays out of
    Git
```

With `src/`, Python can only import your package after you install it (`pip install -e .`). Tests then run against the installed package, exactly as a user or a job would. Without it, tests silently import whatever sits in the working directory, and "works on my machine" follows.

Notebooks explore and present. Functions that matter move into `src/`, where tests and type checks reach them. A notebook that defines the feature pipeline inline cannot be tested or reused by the training job.

Pin `requires-python` to the versions you test, for example `">=3.11"`, so pip refuses an interpreter you never tried.

Why it matters

A team trains a churn model from `train.ipynb`, which defines `build_features()` in a cell. The scoring job copies the function into `score.py`. A month later someone fixes a leakage bug in the notebook version only. Training and scoring now compute features differently, and live accuracy drops 9 points while offline metrics look fine.

With `build_features()` in `src/churn/features.py`, imported by both the notebook and the job, one fix reaches both, and one test protects it.

Worked example: weak code

```
churn/
├── train.ipynb            # defines build_features() in cell 7
├── score.py               # copy of build_features(), edited later
├── utils.py
├── requirements.txt       # pandas, scikit-learn (no versions)
├── setup.cfg              # flake8 settings
└── data.csv               # 2 GB client extract, committed
```

1. **Line 2.** Logic trapped in a notebook: untestable, and copied instead of imported.
2. **Line 3.** Two versions of one function drift apart.
3. **Line 5.** No pins: every install resolves different versions.
4. **Line 7.** Client data in Git: a size problem and a data-protection problem.

Worked example: strong code

```
churn/
├── pyproject.toml         # project, dependency groups, ruff, mypy,
  pytest
├── src/churn/
│   ├── __init__.py
│   ├── features.py        # build_features(), used by training and
  scoring
│   ├── train.py
│   └── score.py
├── tests/
│   └── test_features.py
├── notebooks/
│   └── 01_explore.ipynb   # imports churn.features
```

```
12 └── data/
4      └── sample_200_rows.csv
```

1. **Line 2.** One file configures the project and every tool.
2. **Line 5.** One definition, imported everywhere it is needed.
3. **Line 9.** Tests import the installed package, as the jobs do.
4. **Line 13.** A small, anonymised sample for tests. Real data stays in governed storage.

Logic lives once, in an installable package. Notebooks, jobs and tests import it.

Recap card

- **What goes in pyproject.toml?** [project] with dependencies, [dependency-groups] for development tools, and [tool.*] settings for ruff, mypy and pytest.
- **Install a dependency group** pip install --group dev (pip 25.1 and later).
- **Why the src layout?** Only the installed package is importable, so tests run against it exactly as jobs do.
- **Where does shared logic live?** In the package under src/, imported by notebooks, jobs and tests. Never copied.

Practice

Exercise 1. Multiple choice (level 1)

Where do you declare `pytest` and `ruff` so they install for developers but never ship with the package?

- a) `[project] dependencies`
- b) `[dependency-groups]`
- c) `[tool.pytest.ini_options]`
- d) A comment in README.md

Why?

- r1) Dependency groups (PEP 735) are named lists that are not part of the built distribution.
- r2) Dependency groups are only for test tools.

Exercise 2. Fill the gap (level 2)

Complete the `pyproject.toml` so runtime needs and developer tools are separate.

What you need to know.

A `pyproject.toml` file is written in TOML. A name in square brackets starts a **table**, a section of settings. Each line below it is `key = value`. A list goes in square brackets.

```toml
[project]
name = "survey-tools"
requires-python = ">=3.11"
dependencies = ["numpy>=2.0,<3", "httpx>=0.28"]

[dependency-groups]
dev = ["pytest>=9"]
```

Read it as: the package `survey-tools` needs Python 3.11 or later, and NumPy and httpx to run. The `dev` group lists tools only developers install.

```toml
[project]
name = "churn"
requires-python = ">=3.11"
_____(1) = ["pandas>=3.0,<3.1", "scikit-learn>=1.8,<1.10"]

[_____(2)]
dev = ["pytest>=9", "ruff>=0.16", "mypy>=2.3"]
```

Exercise 3. Write the function (level 3)

Write `unpinned(pyproject_text)`. Parse the TOML with the standard library and return, sorted, every entry in `[project].dependencies` that has no version constraint at all (no `=`, `<`, `>`, `~` or `!`). Return an empty list when there is no dependencies key.

What you need to know.

Pins and constraints. A dependency string is a package name, optionally followed by a version constraint: `numpy>=2.0,<3` allows 2.x only, `requests==2.32.0` pins one version, `pydantic~=2.12` means 2.12 or a later 2.x. A bare name such as `httpx` has no constraint, so every install takes whatever version is newest. That entry is **unpinned**. Constraints use the characters `=`, `<`, `>`, `~` and `!`.

Reading TOML. `tomllib` is in the standard library from Python 3.11. `tomllib.loads(text)` turns TOML text into nested dicts, one per table:

```python
import tomllib

data = tomllib.loads('[project]\nname =
    "x"\ndependencies = ["numpy>=2.0", "httpx"]\n')
data["project"]["dependencies"]   # ['numpy>=2.0', 'httpx']
data.get("tool", {})              # {}: the table is missing
```

Your code must pass these checks:

- numpy and requests have no constraint. Constrained entries must not be listed.
- Without a dependencies key, return an empty list.
- An extra in brackets is not a version constraint. uvicorn[standard] is still unpinned.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
CONSTRAINT_CHARS = ...

# Runtime dependencies with no version constraint.
def unpinned(pyproject_text: str) -> list[str]: ...
```

Starter code:

```python
import tomllib


def unpinned(pyproject_text: str) -> list[str]:
    ...
```

Exercise 4. Multiple choice (level 4)

With a src layout, a colleague runs `pytest` straight after cloning and gets `ModuleNotFoundError: No module named 'churn'`. What is the right fix?

- a) Add `sys.path.insert(0, 'src')` at the top of every test file.
- b) Run `pip install -e .` in the project's virtual environment.
- c) Move the package out of src/ so imports work from the root.
- d) Copy the package into tests/.

Why?

- r1) The src layout is designed so that only the installed package is importable, which makes tests use it like any user would.
- r2) pytest cannot find packages in folders.

Exercise 5. Architecture design (level 5)

Place each piece of a modelling project.

Options: src/<package>/, A notebook cell, tests/, Outside the repository.

1. build_features(), used by training and by the nightly scoring job
2. Plots comparing feature distributions for a one-off question
3. A check that build_features() gives the right output for 5 hand-made rows

4. The full client data extract

Sources

- PEP 735 defines [dependency-groups]; pip 25.1 (April 2025) added --group. (search-only: https://peps.python.org/pep-0735/ and https://ichard26.github.io/blog/2025/04/whats-new-in-pip-25.1/)
- pip 26.2.1 lists --group in `pip install --help`; tomllib is in the standard library from Python 3.11. (verified: Checked in this repository's build environment.)

B1-L2 One virtual environment per project, and python -m pip

Concept

A **virtual environment** is a folder with its own Python interpreter link and its own `site-packages`. Packages you install there do not touch the system Python or any other project.

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -e . --group dev
```

Rules that prevent most "it worked yesterday" problems:

- **One environment per project.** Two projects that share an environment share their version conflicts. Upgrading pandas for one breaks the other.
- **Never install into the system Python.** Operating systems and other tools depend on it.
- **Use `python -m pip`, not bare `pip`.** Bare `pip` runs whichever `pip` comes first on your PATH, which may belong to a different interpreter. `python -m pip` always installs into the interpreter you are about to run.
- **Keep `.venv/` out of Git.** It is large, machine-specific and rebuildable. Commit `pyproject.toml` and a lock file instead (lesson B6-L3).

In code, `sys.prefix != sys.base_prefix` is True inside a virtual environment. A script that must not run against the system Python can check it and stop.

Sometimes one project needs **two** environments, because two dependencies cannot live together. A real case from this course's own build: PySpark 4.0.4 fails to import its pandas integration with pandas 3.x, so the Spark test harness runs in its own environment pinned to pandas 2.x, while the rest

uses pandas 3. Separate environments, each fully pinned, beat one environment held together by guesswork.

Tools such as `uv` create and sync environments much faster, and conda manages non-Python libraries. The principles are the same.

Why it matters

An analyst runs `pip install statsmodels` in a terminal, then `python analysis.py` in the same terminal, and gets `ModuleNotFoundError`. `pip` belonged to one Python, and `python` to another. They try `sudo pip install`, which upgrades NumPy in the system Python and breaks a command-line tool the operating system uses. Two hours are gone.

`python -m venv .venv`, activation, and `python -m pip install` make the interpreter that installs and the interpreter that runs the same one.

Worked example: weak code

```
❶ $ pip install pandas scikit-learn
❷ $ sudo pip install --upgrade numpy
  3 $ python train.py
❸ ModuleNotFoundError: No module named 'sklearn'
❹ $ git add .venv/ data/ train.py
```

1. **Line 1.** Bare pip, no environment: installs into whichever Python owns this pip.
2. **Line 2.** Changes the system Python that the operating system relies on.
3. **Line 4.** The python that runs is not the one pip installed into.
4. **Line 5.** Commits a machine-specific environment folder and data.

Worked example: strong code

```
❶ $ python -m venv .venv
  2 $ source .venv/bin/activate
  3 (.venv) $ python -m pip install --upgrade pip
❷ (.venv) $ python -m pip install -e . --group dev
  5 (.venv) $ python -c "import sys; print(sys.prefix != sys.base_prefix)"
❸ True
  7 (.venv) $ python train.py
```

1. **Line 1.** A project-local environment, listed in .gitignore.
2. **Line 4.** The same interpreter installs and runs. Dependencies come from pyproject.toml.
3. **Line 6.** Proof that the active interpreter is the virtual environment.

One environment per project, created from pyproject.toml, with python -m pip so installing and running use the same interpreter.

Recap card

- **Create and use a project environment** python -m venv .venv, activate it, then python -m pip install -e . --group dev.
- **Why python -m pip?** It installs into the interpreter that python runs. Bare pip may belong to another Python.
- **Detect a virtual environment in code** sys.prefix != sys.base_prefix.
- **Two dependencies that cannot coexist** Two pinned environments and two dependency groups, each tested in CI.

Practice

Exercise 1. Multiple choice (level 1)

Why prefer `python -m pip install x` over `pip install x`?

- a) It is faster.
- b) It installs into the interpreter that `python` runs, whatever is first on PATH.
- c) It installs packages globally for all users.
- d) It skips dependency resolution.

Why?

- r1) `python -m pip` runs pip as a module of that exact interpreter.
- r2) Bare pip is deprecated.

Exercise 2. Predict the output (level 2)

You run this with `.venv/bin/python`, a virtual environment created by `python -m venv .venv`. What does it print?

```
import sys

print(sys.prefix != sys.base_prefix)
```

Exercise 3. Fill the gap (level 2)

Create the environment, then install the project in editable mode with its dev group.

```
python -m _____(1) .venv
source .venv/bin/activate
python -m pip install _____(2) . --group dev
```

Exercise 4. Write the function (level 3)

Write `require_virtualenv()`. It raises `RuntimeError` with the message `"activate the project's virtual environment first"` when the running

interpreter is not inside a virtual environment, and returns `None` otherwise. Read `sys.prefix` and `sys.base_prefix` when the function is called, not at import time.

Your code must pass these checks:

- Inside a venv (prefixes differ) the function must return None.
- Outside a venv (prefixes equal) raise RuntimeError that mentions the virtual environment.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Stop when not running inside a virtual environment.
def require_virtualenv() -> None: ...
```

Starter code:

```python
import sys


def require_virtualenv() -> None:
    ...
```

Exercise 5. Architecture design (level 5)

Your project needs pandas 3.0 for analysis, and a Spark test harness on PySpark 4.0.4, which fails to import its pandas integration with pandas 3.x. Choose the set-up.

Options: Two environments: dev (pandas 3) and spark (pandas 2), each pinned, One environment with the latest of everything, One environment, everything downgraded to pandas 2.

1. Environments
2. Where the difference is recorded
3. CI

Sources

- sys.prefix differs from sys.base_prefix inside a venv. (verified: Python venv documentation; checked in this repository's virtual environments.)
- PySpark 4.0.4 fails to import assertDataFrameEqual with pandas 3.x. (verified: Reproduced in this repository (VERSIONS.md).)

B1-L3 ruff and mypy in daily work

Concept

Two tools catch most mistakes before a test runs.

ruff lints and formats. On a ten-line sample it reported three real problems:

```
m.py:1:1: I001 [*] Import block is un-sorted or un-formatted
m.py:1:8: F401 [*] `os` imported but unused
m.py:9:30: B905 `zip()` without an explicit `strict=` parameter
```

B905 matters for data work: `zip(flags, weights)` silently stops at the shorter list, so a missing weight drops a respondent without an error. `zip(..., strict=True)` raises instead.

mypy checks types. On the same file, strict mode found:

```
m.py:14: error: Unsupported operand types for * ("None" and "int")
    [operator]
m.py:17: error: Function is missing a type annotation  [no-untyped-def]
```

Line 14 multiplies a `float | None` by 100. When the total weight is zero, the function returns `None` and the report crashes. mypy found the zero-weight bug without any test data.

Daily habits:

- Configure both once in `pyproject.toml` (`[tool.ruff]` with the rule sets you want, `[tool.mypy]` with `strict = true`).
- Run `ruff check --fix .` and `ruff format .` before each commit. `--fix` applies only safe fixes.
- Run `mypy` on `src/` and `tests/`. Fix errors by handling the case, not by adding `# type: ignore`.
- Run the same commands in CI (lesson A10-L3), so the rules are not optional.

Libraries without type information show `import-untyped` or missing-stub notes. Install the stub package if one exists, or add a targeted `[[tool.mypy.overrides]]` with `ignore_missing_imports = true` for that module only.

Why it matters

A reporting script formats a weighted share as `f"{share * 100:.1f}%"`. For eleven months every subgroup has respondents. Then a new region with no fieldwork yet appears in the sample frame, its total weight is zero,

`weighted_share` returns `None`, and the monthly client pack fails to build on deadline day.

mypy flags `None * 100` the first time it runs on the file. The fix is two lines, written months before the failure.

Worked example: weak code

```
import os
import pandas as pd


def weighted_share(flags, weights):
    total = sum(weights)
    if total == 0:
        return None
    return sum(w for f, w in zip(flags, weights) if f == 1) / total


def report(flags, weights):
    return f"{weighted_share(flags, weights) * 100:.1f}%"
```

1. **Line 1.** F401: unused import.
2. **Line 5.** No hints: mypy cannot see that the function can return None.
3. **Line 9.** B905: zip silently truncates to the shorter list.
4. **Line 13.** Crashes with TypeError when the share is None.

Worked example: strong code

```
def weighted_share(flags: list[int], weights: list[float]) -> float | None:
    """Weighted share of flagged rows, or None when the total weight is 0."""
    total = sum(weights)
    if total == 0:
        return None
    return sum(w for f, w in zip(flags, weights, strict=True) if f == 1) / total


def report(flags: list[int], weights: list[float]) -> str:
    """Share as a percentage string, or 'n/a' when undefined."""
    share = weighted_share(flags, weights)
    if share is None:
        return "n/a"
    return f"{share * 100:.1f}%"
```

1. **Line 1.** The return type admits None, so mypy makes every caller handle it.
2. **Line 6.** Unequal lengths raise ValueError instead of dropping rows.
3. **Line 12.** The case mypy flagged, handled on purpose.

ruff and mypy found an unused import, a silent truncation and a crash on zero weight, before anyone wrote a test.

Recap card

- **Why does ruff flag zip() without strict= (B905)?** zip silently truncates to the shorter input. strict=True raises ValueError instead.
- **mypy says None * int is unsupported. The fix?** Handle the None case in code. Never # type: ignore it away.
- **Daily commands before a commit** ruff check --fix ., ruff format ., mypy.
- **Missing stubs for one library** Install types-<lib> if it exists, otherwise a per-module mypy override. Never global.

Practice

Exercise 1. Predict the output (level 1)

What does this print?

```
flags = [1, 0, 1]
weights = [2.0, 1.0]
print(list(zip(flags, weights)))
```

Exercise 2. Multiple choice (level 2)

mypy reports `Unsupported operand types for * ("None" and "int")` on `share * 100`. What is the right fix?

- a) Add `# type: ignore` to the line.
- b) Handle the None case before the multiplication.
- c) Change the return type to `float` and keep returning None.
- d) Turn off strict mode.

Why?

- r1) mypy found a real path where the value is None. Handling it removes the crash.
- r2) mypy errors are style issues.

Exercise 3. Fill the gap (level 2)

Complete the pre-commit routine: apply safe lint fixes, format, then type-check.

```
ruff check _____(1) .
ruff _____(2) .
_____(3)
```

Exercise 4. Refactor (level 4)

Make this pass `ruff` (no B905) and `mypy --strict`: add type hints, keep the None result for zero weight, make unequal lengths raise, and make `report` return `"n/a"` instead of crashing.

Your code must pass these checks:

- The share of weight 3 out of 4 is 0.75.
- Zero weight: weighted_share returns None and report returns 'n/a'.
- report must format the share as a percentage with one decimal: 75.0%.
- Unequal lengths must raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Weighted share of flagged rows, or None when the total weight is 0.
def weighted_share(
    flags: list[int],
    weights: list[float],
) -> float | None: ...

# Share as a percentage string, or 'n/a' when undefined.
def report(flags: list[int], weights: list[float]) -> str: ...
```

Starter code:

```python
def weighted_share(flags, weights):
    total = sum(weights)
    if total == 0:
        return None
    return sum(w for f, w in zip(flags, weights) if f == 1) / total


def report(flags, weights):
    return f"{weighted_share(flags, weights) * 100:.1f}%"
```

Exercise 5. Architecture design (level 5)

mypy reports `Skipping analyzing "shapely": module is installed, but missing library stubs or py.typed marker`. Choose the response for each case.

Options: Install the stubs in the dev group, Add [[tool.mypy.overrides]] with ignore_missing_imports for that module, Set ignore_missing_imports = true globally.

1. A stub package, types-shapely, exists on PyPI
2. No stubs exist for an internal library
3. A colleague proposes silencing all such messages for every library

Sources

- ruff reports I001, F401 and B905 and mypy --strict reports [operator] and [no-untyped-def] on the sample shown. (verified: ruff 0.16.9 and mypy 2.3.1 run in this repository.)
- zip(strict=True) raises ValueError on unequal lengths (Python 3.10+). (verified: Python standard library; run in Python 3.11.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B1-L1 One pyproject.toml and a src layout

B1-L1 exercise 1

Hints. (1) Which table never ships in the built package? (2) It was added by PEP 735. (3) `[dependency-groups]`.

Answer. b) `[dependency-groups]` Because: r1) Dependency groups (PEP 735) are named lists that are not part of the built distribution.

`[dependency-groups]` holds development-only requirements. `pip install --group dev` (pip 25.1+) installs them.

Common wrong answers.

- `a`: `[project] dependencies` install for every user of the package, in production too.
- `c`: That table configures how pytest runs. It does not install anything.
- `d`: A comment installs nothing and drifts from reality.
- `b|r2`: Right table, wrong reason. Groups can hold any development tools: tests, linting, docs, notebooks.

B1-L1 exercise 2

Hints. (1) The runtime key is plural. (2) The development table has a hyphenated name. (3) `dependencies` and `dependency-groups`.

Answer.

```
[project]
name = "churn"
requires-python = ">=3.11"
dependencies = ["pandas>=3.0,<3.1", "scikit-learn>=1.8,<1.10"]

[dependency-groups]
dev = ["pytest>=9", "ruff>=0.16", "mypy>=2.3"]
```

`[project].dependencies` lists what users need to run the package. `[dependency-groups]` lists what developers need to work on it.

Common wrong answers.

- g1 = `requires`: The runtime list under [project] is called `dependencies`.

- g2 = `tool.dev`: `[tool.*]` tables configure tools. Development requirements go in `[dependency-groups]`.
- g2 = `optional-dependencies`: Optional dependencies (extras) ship with the package for users to opt into. Development-only tools belong in `[dependency-groups]`.

B1-L1 exercise 3

Hints. (1) `tomllib.loads(text)` returns a dict. (2) Use `.get` with defaults for the project table and the dependencies list. (3) Keep entries that contain none of = < > ~ !.

Answer.

```python
import tomllib

CONSTRAINT_CHARS = set("=<>~!")


def unpinned(pyproject_text: str) -> list[str]:
    """Runtime dependencies with no version constraint.

    Args:
        pyproject_text: Contents of pyproject.toml.

    Returns:
        Sorted requirement strings without =, <, >, ~ or !.
    """
    project = tomllib.loads(pyproject_text).get("project", {})
    deps = project.get("dependencies", [])
    return sorted(d for d in deps if not CONSTRAINT_CHARS & set(d))
```

`tomllib` ships with Python 3.11+. Checking characters is enough to spot a missing constraint, and `.get` handles projects that declare no dependencies.

B1-L1 exercise 4

Hints. (1) Why can Python not see src/churn? (2) What makes a package importable everywhere in the environment? (3) An editable install.

Answer. b) Run `pip install -e .` in the project's virtual environment. Because: r1) The src layout is designed so that only the installed package is importable, which makes tests use it like any user would.

`pip install -e .` installs the package in editable mode, so imports resolve to `src/churn` and edits take effect without reinstalling.

Common wrong answers.

- `a`: Path hacks in every file defeat the point of the layout and break when files move.

- `c`: That removes the protection the layout gives: tests would import the working directory instead of the installed package.
- `d`: Copies drift. Tests must exercise the real package.
- `b|r2`: Right fix, wrong reason. pytest can import from folders. The src layout deliberately hides the package until it is installed.

B1-L1 exercise 5

Hints. (1) What must more than one program import? (2) What should CI run on every change? (3) What must never be cloned?

Answer.

- Shared logic in the package, imported by notebook and jobs.
- One-off exploration in notebooks, which import the package.
- Known-answer checks in tests/.
- Real data in governed storage, never in Git. Keep only small anonymised samples in the repository.

Package for shared logic, notebooks for exploration, tests for checks, governed storage for data.

Common wrong answers.

- features = `notebook`: The scoring job cannot import a notebook cell, so it would copy it, and the copies would drift.
- extract = `src`: Client data in the repository leaks with every clone. Keep it in governed storage.
- known_answer = `notebook`: A check in a notebook runs only when someone reruns the notebook. Put it in tests/ so CI runs it.

B1-L2 One virtual environment per project, and python -m pip

B1-L2 exercise 1

Hints. (1) How many Pythons might be on your PATH? (2) Which pip does `python -m pip` use? (3) The one belonging to that python.

Answer. b) It installs into the interpreter that `python` runs, whatever is first on PATH. Because: r1) `python -m pip` runs pip as a module of that exact interpreter.

`python -m pip` ties the install to the interpreter you will run. Bare `pip` is whichever pip script PATH finds first.

Common wrong answers.

- `a`: Speed is the same. The benefit is installing into the right interpreter.
- `c`: It installs into the current interpreter's environment, which inside a venv is the project only.
- `d`: Resolution is identical. Only the target interpreter is guaranteed.
- `b|r2`: Right answer, wrong reason. Bare pip still works. It may simply belong to a different Python.

B1-L2 exercise 2

Hints. (1) What does sys.prefix point to inside a venv? (2) And sys.base_prefix? (3) They are different folders.

Answer.

```
1 True
```

`sys.prefix` is the virtual environment, `sys.base_prefix` is the Python it was created from. They differ only inside a venv.

Common wrong answers.

- `False`: Inside a venv, sys.prefix points at the venv folder and sys.base_prefix at the base installation, so they differ.

B1-L2 exercise 3

Hints. (1) The standard library module that creates environments. (2) The flag for an editable install. (3) `venv` and `-e`.

Answer.

```
1 python -m venv .venv
2 source .venv/bin/activate
3 python -m pip install -e . --group dev
```

`python -m venv .venv` creates the environment. `pip install -e .` installs the project so edits under src/ take effect immediately.

Common wrong answers.

- g1 = `virtualenv`: virtualenv is a separate package. The standard library module is `venv`.
- g2 = `-r`: `-r` reads a requirements file. Editable installs of the project use `-e .`.

B1-L2 exercise 4

Hints. (1) Inside a venv the two prefixes differ. (2) Raise when they are equal. (3) Read them inside the function body.

Answer.

```python
import sys


def require_virtualenv() -> None:
    """Stop when not running inside a virtual environment.

    Raises:
        RuntimeError: If sys.prefix equals sys.base_prefix.
    """
    if sys.prefix == sys.base_prefix:
        raise RuntimeError("activate the project's virtual environment
    first")
```

Equal prefixes mean the system or base interpreter is running. Reading them at call time keeps the check correct in tests and in any process that changes them.

B1-L2 exercise 5

Hints. (1) Can one environment hold both constraints? (2) Where will the next developer look? (3) CI must match how each environment is used.

Answer.

- Two environments, because the constraint is real and tested.
- Two dependency groups in pyproject.toml, with the reason written next to the pin.
- Two CI jobs, so each environment is tested exactly as it is used.

When two tools conflict, give each its own pinned environment, record why in pyproject.toml, and test both in CI.

Common wrong answers.

- envs = `one_latest`: PySpark 4.0.4 fails with pandas 3.x: `assertDataFrameEqual` raises ImportError. One environment cannot satisfy both.
- envs = `one_old`: That works, but the analysis code then runs on an older pandas than intended. Separate environments keep each part current.
- ci = `one_job`: Installing both groups together recreates the conflict in CI.
- declare = `readme`: Chat messages are lost. Record the pin and its reason in pyproject.toml.

B1-L3 ruff and mypy in daily work

B1-L3 exercise 1

Hints. (1) How long is each list? (2) What does zip do when lengths differ? (3) It stops at the shortest.

Answer.

```
1 [(1, 2.0), (0, 1.0)]
```

`zip` stops at the shortest input, so the third respondent disappears. ruff's B905 asks for `strict=True`, which raises `ValueError` on unequal lengths (Python 3.10+).

Common wrong answers.

- `ValueError`: Only `zip(..., strict=True)` raises. Plain zip stops at the shorter input without a word.
- `[(1, 2.0), (0, 1.0), (1, None)]`: That is `itertools.zip_longest`. zip drops the extra element.

B1-L3 exercise 2

Hints. (1) When can share be None? (2) Is that case possible in real data? (3) Handle it.

Answer. b) Handle the None case before the multiplication. Because: r1) mypy found a real path where the value is None. Handling it removes the crash.

mypy found a reachable crash. Handle `None` explicitly, for example by returning `"n/a"`.

Common wrong answers.

- `a`: The ignore hides the warning and keeps the crash.
- `c`: The hint would then lie, and mypy would flag the `return None` instead. The crash stays.
- `d`: Weakening the checker hides this bug and every future one like it.
- `b|r2`: Right fix, wrong reason. This is a real crash on zero weight, not style.

B1-L3 exercise 3

Hints. (1) The flag that applies safe fixes. (2) The formatter subcommand. (3) The type checker's command.

Answer.

```
ruff check --fix .
ruff format .
mypy
```

`ruff check --fix .` applies safe fixes, `ruff format .` formats, and `mypy` type-checks with the settings in pyproject.toml.

Common wrong answers.

- g1 = `--unsafe-fixes`: Unsafe fixes can change behaviour. Use --fix for the safe ones, and review the rest by hand.
- g3 = `python -m pytest`: Tests matter, but the step after formatting in this routine is the type checker.

B1-L3 exercise 4

Hints. (1) Annotate: `flags: list[int], weights: list[float]`. (2) `zip(flags, weights, strict=True)`. (3) In report, store the share and return 'n/a' when it is None.

Answer.

```python
def weighted_share(flags: list[int], weights: list[float]) -> float | None:
    """Weighted share of flagged rows, or None when the total weight is 0."""
    total = sum(weights)
    if total == 0:
        return None
    return sum(w for f, w in zip(flags, weights, strict=True) if f == 1) / total


def report(flags: list[int], weights: list[float]) -> str:
    """Share as a percentage string, or 'n/a' when undefined."""
    share = weighted_share(flags, weights)
    if share is None:
        return "n/a"
    return f"{share * 100:.1f}%"
```

The types make the None case visible, `strict=True` stops silent truncation, and report handles None on purpose.

B1-L3 exercise 5

Hints. (1) Prefer real type information. (2) When there is none, narrow the exception. (3) Never global.

Answer.

- Stubs exist: install them in the dev group, and mypy checks your use of the library.

- No stubs: silence only that module with a targeted override.
- Never switch it off globally, or you lose the check for libraries that do ship types.

Install stubs where they exist, override per module where they do not, and keep the global check on.

Common wrong answers.

- stubs_exist = `override`: Stubs exist, so installing them gives you real checks for free.
- many = `global`: A global setting hides missing stubs you could have installed, for every library, forever.